

Receipt Fraud Detection

Stockwell Solar — Expense Tracking System

Prepared: June 30, 2026

Problem Statement

Employees submit receipt images when adding expenses. Fraud risks include:

- Same receipt submitted in multiple months (reuse attack)
- Receipt image slightly edited (amount changed, date changed) and resubmitted
- Receipts downloaded from the internet or generated by AI
- Screenshots of bills instead of original photos
- Receipts from another person shared and submitted

Detection Strategy — 4 Signal Layers

Signals are layered from cheapest to most expensive. Layer 1 runs on every upload. Layers 3–4 are optional and run only when needed.

Layer	Signal	What it detects	Cost
1	Exact Hash (SHA-256)	Identical file submitted again (zero-edit reuse)	Near zero
2	EXIF Timestamp (DateTimeOriginal)	Photo taken date vs. expense date mismatch (e.g. 2025 photo used for 2026 expense)	Very low
3	EXIF Software Tag	Image was edited in Photoshop / Lightroom / Screenshot tool — not a direct camera photo	Very low
4	Missing / Sparse EXIF	No camera metadata = screenshot, web download, or AI-generated image	Very low

Layer 1 — Exact Hash (SHA-256)

When a receipt is uploaded, compute its SHA-256 hash and store it in the database alongside the expense record. On each new upload, check if that hash already exists.

Catches: Employee uploads exact same file for March expense and April expense.

Misses: Even 1 pixel change or file re-save defeats this check. Works only for zero-edit reuse.

Implementation (Node.js / Supabase):

1. On upload: `crypto.createHash('sha256').update(fileBuffer).digest('hex')`
2. Store in expenses table: `receipt_hash` column (text)
3. Before saving: `SELECT id FROM expenses WHERE receipt_hash = $hash AND user_id = $uid`
4. If row found → warn admin and employee

Layer 2 — EXIF Timestamp (DateTimeOriginal)

Camera photos embed the capture date inside the image file as EXIF metadata. We read DateTimeOriginal and compare it against the expense date (month_year field).

Catches: Employee took a photo in January 2025, submits it for June 2026 expense.

Misses: Employee can manually edit EXIF metadata with free tools. Not a definitive proof — flag for admin review, not auto-reject.

Flag condition:

If DateTimeOriginal year/month differs from expense month_year by more than 1 month → flag as 'Date mismatch: photo taken [date], expense claimed [date]'

Layer 3 — EXIF Software Tag

When an image is opened and re-saved by editing software, that software writes its name into the EXIF 'Software' field. Screenshots also leave a signature.

Software Tag value	What it means
Adobe Photoshop	Image was edited in Photoshop
Lightroom	Image was edited in Lightroom
Microsoft Paint	Opened and saved in MS Paint
Snagit / ShareX / Snipping Tool	Screenshot taken on PC
WhatsApp / Telegram	Downloaded from messaging app
■■■■ (699pic.com) / Getty / Shutterstock	Downloaded from stock photo site
GIMP	Edited in free photo editor

Real test result from our data: example.jpg had Artist='■■■■' and Copyright='http://699pic.com' — a Chinese stock photo website. Clearly not an original expense receipt.

Layer 4 — Missing / Sparse EXIF

A genuine camera photo has rich EXIF: camera make/model, focal length, ISO, aperture, GPS. If EXIF is completely absent or contains only 2-3 basic fields, the image is likely a screenshot, web download, or AI-generated image.

Image type	EXIF fields present	Flag?
Camera photo (original)	20–40 fields (Make, Model, ISO, GPS...)	No
Screenshot (Windows/Mac)	2–5 fields (JFIF only)	Yes — flag
WhatsApp compressed image	5–10 fields (stripped)	Maybe — soft flag
AI-generated image	0 fields or PNG header only	Yes — flag
Stock photo download	Has fields but Artist/Copyright suspicious	Yes — flag

Real test results: test.jpg had only sparse JFIF fields (screenshot). test2.png had zero EXIF and 1536x1024 resolution — classic AI-generated image dimensions.

Optional / Advanced Signals (Future)

Signal	What it detects	Cost	Priority
GPS Location Check	Photo GPS vs. employee site location mismatch	Low	Medium
OCR Date Cross-check	Date printed on receipt vs. EXIF date vs. expense date	Medium	High
Perceptual Hash (pHash)	Near-duplicate images (cropped, resized, watermarked)	Medium	Medium
Error Level Analysis (ELA)	Edited/tampered regions in JPEG images	High	Low

Implementation Plan

Step 1 — Add receipt_hash column to Supabase

```
ALTER TABLE expenses ADD COLUMN receipt_hash TEXT;  
ALTER TABLE expenses ADD COLUMN exif_flags JSONB; -- stores all flag data
```

Step 2 — On receipt upload (add-expense.html)

- a) Compute SHA-256 of file bytes (client-side with Web Crypto API)
- b) Send hash + file to existing receipt-url.js API
- c) API checks for duplicate hash in expenses table
- d) API reads EXIF from image (using exifr npm package)
- e) API builds exif_flags object and returns to client

Step 3 — Flag result handling

- If any flag triggered:
- Store flags in exif_flags column
 - Show warning banner on add-expense form (not a hard block)
 - Admin sees flag indicators in dashboard approval modal

Step 4 — Admin view

- In dashboard expense approval modal:
- Show flag badges: [DUPLICATE HASH] [DATE MISMATCH] [EDITED] [NO EXIF]
 - Admin can still approve/reject — flags are advisory, not automatic

Flag Severity Guide for Admins

Flag	Severity	Action recommended
------	----------	--------------------

Duplicate hash	HIGH	Almost certainly reuse — reject and ask employee
Photo date > 2 months before expense	MEDIUM	Ask employee for explanation
Software tag = Photoshop / GIMP / Paint	HIGH	Image was edited — reject unless explained
Software tag = WhatsApp / Telegram	LOW	Photo forwarded via chat — common, acceptable
Software tag = stock photo site	HIGH	Not employee's own photo — reject
Missing EXIF (screenshot / AI)	MEDIUM	Ask for original camera photo
Sparse EXIF only	LOW	May be WhatsApp compression — soft flag

Limitations

- A determined fraudster can strip EXIF metadata or manually set it — these signals are not foolproof.
- WhatsApp and messaging apps compress images and strip EXIF — legitimate receipts may be flagged.
- OCR-based date cross-check requires an OCR library (Tesseract) which adds server complexity.
- Perceptual hash and ELA require Python services (not native to Node.js/Vercel serverless).
- These are advisory signals — final decision always rests with the Admin/Super Admin approver.